ResearchGPT

Project Documentation

An agentic Retrieval-Augmented Generation system that transforms a research topic into a fully formatted IEEE research paper or literature survey — powered by hybrid search, cross-encoder reranking, and LLM-based generation and evaluation.

Table of Contents

1. **Overview**
2. **Architecture**
3. **UI Features**
4. **Backend Pipeline**
   - 4.1 Ingestion
   - 4.2 Processing & Chunking
   - 4.3 Embedding & Vector Store
   - 4.4 Retrieval
   - 4.5 Generation
   - 4.6 Evaluation
   - 4.7 Analysis
   - 4.8 Output Generation
   - 4.9 History & Caching
5. **Configuration Reference**
6. **How to Run**

1. Overview

ResearchGPT is an agentic RAG system that fetches real academic papers from arXiv, retrieves relevant content using hybrid search, generates grounded IEEE papers via an LLM, and evaluates output quality — all through an interactive Streamlit web interface.

- Fetches academic papers directly from arXiv based on the user's research topic
- Generates full IEEE-formatted research papers with proper citations
- Produces structured literature surveys with comparative analysis across papers
- Evaluates output quality using G-Eval (Faithfulness & Relevance, scored 1–5)
- Detects plagiarism at sentence level with n-gram overlap analysis
- Supports export to PDF, Word (.docx), CSV, and Excel
- Allows iterative paper refinement through natural language instructions

2. Architecture

The system follows a modular pipeline where each stage is independently configurable:

Stage	Component	Description
1	Ingestion	Fetch & download PDFs from arXiv
2	Processing	Extract text, clean, detect sections, chunk
3	Embedding	Encode chunks with BAAI/bge-base-en-v1.5 → ChromaDB
4	Retrieval	Hybrid BM25 + Vector Search → Cross-Encoder Rerank
5	Generation	LLM synthesizes IEEE paper from retrieved context
6	Evaluation	G-Eval scoring, ROUGE-L, plagiarism detection
7	Output	Export to IEEE PDF, Word (.docx), CSV/Excel

3. UI Features

3.1 Session Management

- Recent Research sidebar — lists all past topics with one-click reload
- New Research button — creates an isolated chat session per topic
- Persistent history — papers and surveys reload from disk between sessions

3.2 Settings Panel

Setting	Options	Default
Author Name / Affiliation / Email	Free text	—
Retrieval Strategy	auto, hyde, complex, standard	auto
Survey Paper Count	3–15	10
Top-K Retrieval Chunks	3–12	5
Refresh from arXiv	Toggle	Off
Run Quality Evaluation	Toggle	On

3.3 Tab 1 — Paper

- Full IEEE-formatted paper with section navigation anchors
- Faithfulness quality badge (Good / Fair / Poor) in the header
- Quick-edit chips: Lengthen, Deeper Methodology, Strengthen Results, Expand Future Work, Formal IEEE Tone, Rewrite Abstract
- PDF and Word (.docx) download buttons
- Section-to-source attribution — which papers contributed to each section
- Citation rebuild tool — reconstructs citations from source metadata

3.4 Tab 2 — Survey & Analysis

- Structured comparison table across 8 fields per paper: Problem Statement, Objective, Research Gap, Methodology, Features, Results & Discussion, Limitations, Future Scope
- Toggle between Table view and Card view
- Direct arXiv links per paper
- Research Gap summary panel
- Export as CSV or Excel

3.5 Tab 3 — Quality & Sources

- Gauge charts for Faithfulness and Relevance (1–5 scale)
- LLM-generated reasoning behind each quality score
- Pipeline stats: paper count, strategy used, timing breakdown (index / retrieval / generation / total)
- Retrieval analytics: chunks retrieved, BM25 vs vector blend ratio, min/avg/max scores
- Paper contribution chart — chunks per source paper
- Retrieved chunks viewer with individual scores
- Plagiarism report: sentence-level similarity % and n-gram overlap %

4. Backend Pipeline

4.1 Ingestion

File: `rag_new/src/ingestion/ingestor.py`

- Fetches papers from arXiv via the arxiv Python library with a direct Atom API fallback
- Exponential backoff retries on failures: 2 → 4 → 8 → 16 seconds
- Rate limiting between downloads (1s default) to respect arXiv limits
- Validates downloaded files (>1000 bytes minimum to detect failed downloads)
- Falls back to arXiv HTML version if PDF is unavailable
- Persists metadata to _papers_metadata.json to skip re-downloads on repeat runs

Parameter	Default	Purpose
MAX_PAPERS	5 (fast) / 8 (full)	Papers fetched per topic
MAX_RETRIES	3	Download retry attempts
RETRY_DELAY	2.0s	Initial backoff (doubles each retry)
DOWNLOAD_RATE_LIMIT	1.0s	Delay between downloads

4.2 Processing & Chunking

File: `rag_new/src/processing/processor.py`

- PDF extraction via PyMuPDF (fitz) — handles text, tables, and multi-page documents
- Cleans junk lines: page numbers, running headers, arXiv/proceedings boilerplate
- Chunking via LangChain RecursiveCharacterTextSplitter with hierarchical separators
- Detects dominant section per chunk (Abstract, Introduction, Methodology, etc.)
- Propagates full paper metadata to every chunk for traceability

Parameter	Value
CHUNK_SIZE	1000 chars
CHUNK_OVERLAP	350 chars (35% overlap for context continuity)
Separators	\n\n → \n → . → (space) → (empty)

4.3 Embedding & Vector Store

File: `rag_new/src/embedding/vector_store.py`

- Embedding model: BAAI/bge-base-en-v1.5 (sentence-transformers)
- Vector database: ChromaDB, persistent and disk-backed
- One collection per research topic — prevents cross-topic noise
- Operations: upsert, semantic query, reset, bulk retrieval for BM25 fitting

- Automatic pruning of stale collections (keeps 10 most recent)

4.4 Retrieval

File: `rag_new/src/retrieval/retriever.py`

All strategies use Hybrid BM25 + Vector Search with strategy-specific blending ratios, followed by Cross-Encoder reranking.

Strategy	How It Works	BM25 Weight
HyDE	LLM generates a hypothetical answer; that answer is embedded and searched	30%
Complex	Query decomposed into 2–3 sub-questions; retrieved independently and merged	55%
Standard	Direct embedding of the raw query + BM25	50%
Auto	Automatically selects strategy based on query characteristics	Varies

Cross-Encoder Re-ranking

- Model: ms-marco-MiniLM-L-12-v2
- Rescores top-20 candidate chunks with (query, document) pairs
- Section-aware bonus (+0.1) for Results, Methodology, Conclusion, Discussion, Experiment, Evaluation, Analysis, Findings
- Returns final top-K after reranking (default: 5)

4.5 Generation

Files: `rag_new/src/generation/llm_client.py`, `rag_new/src/generation/generator.py`

LLM Provider Support

Provider	Mode	Endpoint
Cerebras / OpenAI	Cloud	https://api.cerebras.ai/v1
Ollama	Local	http://localhost:11434/api/generate

Generation Features

- Context refinement: extracts only query-relevant sentences from each chunk before prompting
- Citation validation: removes hallucinated [N] citations not present in actual sources
- Generates full IEEE papers — 8 sections, roman numeral headings, numbered citations
- Exponential backoff on rate limits and timeouts (up to 4 retries)
- Streaming support for long-form generation

Parameter	Default	Purpose
LLM_TEMPERATURE	0.2	Low temperature for consistent, factual output
LLM_MAX_TOKENS	8192	Maximum generation length

Parameter	Default	Purpose
MAX_CONTEXT_TOKENS	3000 chars	Context window per chunk
SECTION_MIN_WORDS	150	Minimum words per paper section
SECTION_MAX_WORDS	600	Target maximum words per section

4.6 Evaluation

File: `rag_new/src/evaluation/evaluator.py`

LLM-Based Metrics (G-Eval)

Metric	Scale	Question Asked
Faithfulness	1–5	Are all claims supported by retrieved context?
Relevance	1–5	Does the paper address the specific research query?

Traditional Metrics

Metric	Description
ROUGE-L	F1 score between generated text and source chunks (LCS-based)
Hit Rate	Whether retrieved chunks include the expected relevant papers
MRR	Mean Reciprocal Rank — position of first relevant result (higher is better)

Plagiarism Detection

- Sentence-level cosine similarity scoring (threshold: 0.65)
- 8-gram overlap detection across all source chunks
- Reports: overall similarity %, max sentence similarity %, n-gram overlap %
- Flags suspicious sentences with matched source and similarity score

4.7 Analysis

File: `rag_new/src/analysis/paper_analyzer.py`

- Fetches paper metadata and abstracts directly from arXiv Atom API
- Downloads and extracts PDF text (up to 10 pages per paper for speed)
- Falls back to abstract only if PDF extraction fails
- LLM extracts 8 structured fields per paper via a single structured prompt
- 1-second delay between LLM calls to respect rate limits
- Generates a sample IEEE overview paper from the combined survey data

Fields extracted per paper:

1. Problem Statement
2. Objective
3. Research Gap
4. Methodology (Algorithms/Techniques)
5. Features (Focused)
6. Results & Discussion (with metrics)
7. Limitations
8. Future Scope

4.8 Output Generation

Directory: `rag_new/src/output_pdf/`

Format	Library	Notes
IEEE PDF	ReportLab	Two-column layout, proper typography and fonts
LaTeX → PDF	ReportLab + pdflatex	Full LaTeX template with equations and tables
Word (.docx)	python-docx	Editable, preserves IEEE section structure
CSV / Excel	pandas	Survey analysis table for data analysis
Survey PDF	ReportLab	Combined analysis table + sample overview paper

4.9 History & Caching

File: `rag_new/src/history/paper_history.py`

- Results persisted to disk, keyed by topic + strategy + fast_mode hash
- Saved data: paper text, evaluation scores, retrieved context, survey rows, timing metrics
- In-process cache with 1-hour TTL — skips full pipeline on repeated identical queries
- Base64-encoded PDFs embedded in history JSON for portability
- Load and delete operations for session management

5. Configuration Reference

All settings are centralized in `rag_new/config/config.py` and can be overridden via a `.env` file at the project root.

Category	Parameter	Default	Description
Ingestion	MAX_PAPERS	5	Papers fetched from arXiv per topic
Ingestion	MAX_RETRIES	3	Download retry attempts
Ingestion	RETRY_DELAY	2.0s	Initial backoff (doubles each retry)
Ingestion	DOWNLOAD_RATE_LIMIT	1.0s	Delay between downloads
Chunking	CHUNK_SIZE	1000	Characters per chunk
Chunking	CHUNK_OVERLAP	350	Overlap chars for context continuity
Embedding	EMBEDDING_MODEL	BAAI/bge-base-en-v1.5	Semantic encoder model
Retrieval	CROSS_ENCODER_MODEL	ms-marco-MiniLM-L-12-v2	Reranker model
Retrieval	TOP_K	5	Final results returned after reranking
Retrieval	RETRIEVAL_WINDOW_SIZE	20	Candidates before reranking
Retrieval	BM25_ALPHA_HYDE	0.7	Vector blend weight for HyDE strategy
Retrieval	BM25_ALPHA_COMPLEX	0.45	Vector blend weight for Complex strategy
Retrieval	BM25_ALPHA_STANDARD	0.5	Vector blend weight for Standard strategy
Generation	LLM_PROVIDER	openai	LLM provider: openai or ollama
Generation	OPENAI_MODEL	gpt-oss-120b	Model ID for cloud generation
Generation	LLM_TEMPERATURE	0.2	Low = consistent and factual output
Generation	LLM_MAX_TOKENS	8192	Maximum tokens in LLM output
Survey	SURVEY_DEFAULT_PAPERS	10	Default papers for literature survey
Survey	SURVEY_MIN_PAPERS	3	Minimum papers allowed in survey
Survey	SURVEY_MAX_PAPERS	15	Maximum papers allowed in survey
Evaluation	PLAGIARISM_THRESHOLD	0.65	Sentence similarity alarm threshold
Vector DB	MAX_STALE_COLLECTIONS	10	Max topic collections to retain on disk

6. How to Run

Start the Web App

```
cd rag_new   streamlit run ui.py
```

CLI Commands

Command	Description
run_all --query "topic"	Run the full end-to-end pipeline
ingest --query "topic"	Download papers from arXiv only
index	Build the vector index only
retrieve --query "topic"	Run retrieval only and print chunks
generate --topic "topic"	Generate paper only (requires existing index)
evaluate --query "topic"	Run evaluation metrics only

Programmatic Usage

```
from src.pipeline.ui_pipeline import run_rag_for_ui_pipeline

result = run_rag_for_ui_pipeline(
    topic="Finetuning methods for large language models",
    retrieval_strategy="hyde",
    include_evaluation=True,
    fast_mode=True,
)

print(result["paper_text"])    # Full IEEE paper
print(result["faithfulness"])  # 1-5 score
print(result["relevance"])     # 1-5 score
```

LLM Provider Configuration (.env)

```
# Use Cerebras / OpenAI (cloud)
LLM_PROVIDER=openai
OPENAI_API_KEY=your_key_here

# OR use Ollama (local)
LLM_PROVIDER=ollama
```